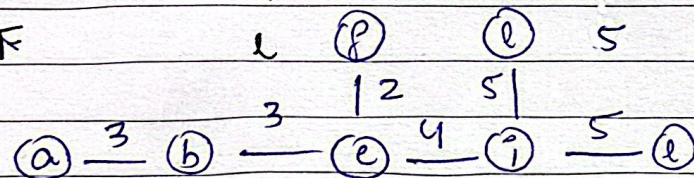


Savita Sangrez
23i-2088
C4-A

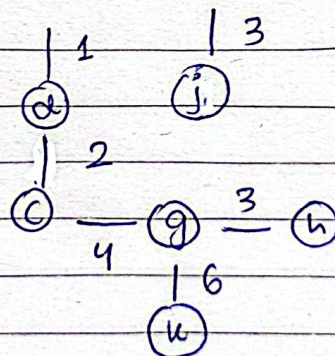
Design And Analysis of Algorithms

Assignment 03

Q1.	Visited	Vertex	Distance	Parent
a)				
①	TF	a	-	-
②	TF	b	3	a
③	TF	c	5	a
④	TF	d	4	e
⑤	TF	e	3	b
⑥	TF	f	2	e
⑦	TF	g	4	c
⑧	TF	h	3	g
⑨	TF	i	4	e
⑩	TF	j	3	i
⑪	TF	k	6	g
⑫	TF	l	5	i



Final cost = 36

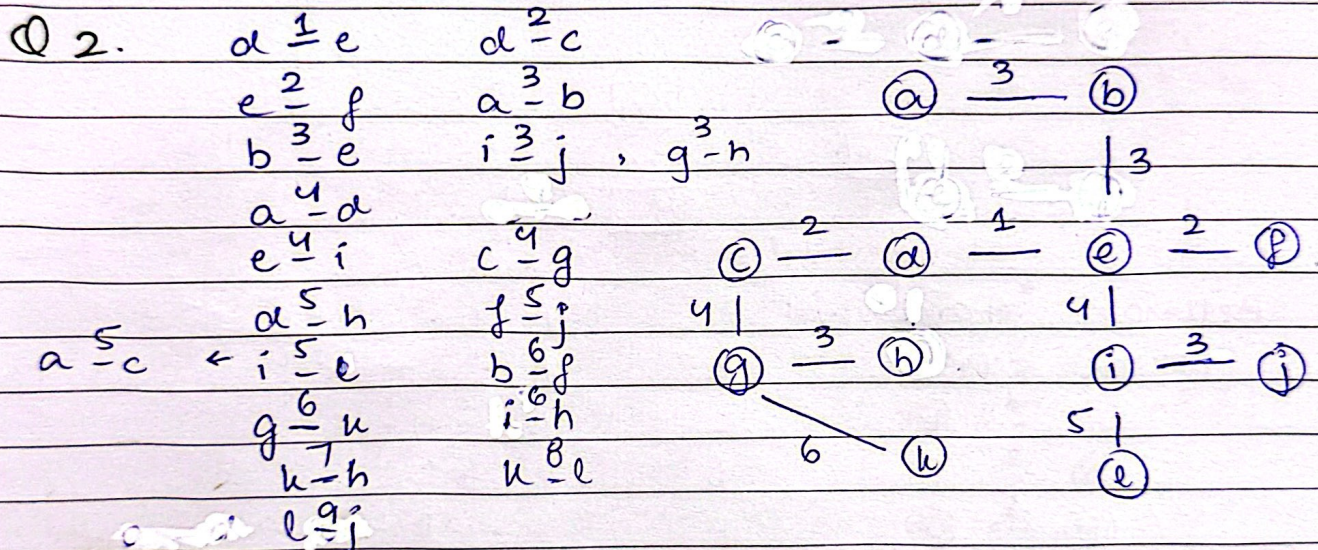


b) Prim's does not ensure connectivity by itself. It only grows a tree from the starting vertex. If graph is not connected, Prim's algorithm will stop and not cover all vertices. So we have to make sure that the graph is connected before applying Prim's.

c) Yes, we can construct a minimum spanning tree of G_{new} by adding one of the new edges in T .

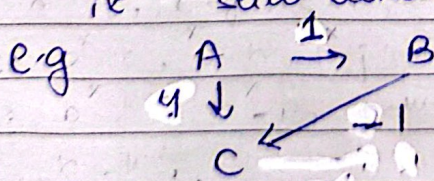
T is already the MST of original graph G , so the only thing we have to do is connect the new vertex to tree.

To keep tree's weight minimum, we will choose the new edge with the smallest weight among all edges connecting the new vertex to vertices in G .



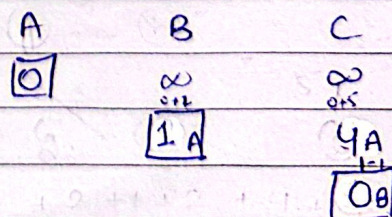
$a-d, a-c, f-j, b-f, k-h, i-h, d-h$
Minimum spanning cost = 36

Q 3. b) No, Dijkstra does not always give correct results on -ve weight cycle. But, in some cases it still works correctly.



we want $A \rightarrow C$

$= 0$ correct as shortest path.



11

 $4+2 \times 5$

A: A J:

B: $A \rightarrow I \rightarrow B$

C: $A \rightarrow H \rightarrow C$

D: $A \rightarrow D$

E: $A \rightarrow I \rightarrow B \rightarrow G \rightarrow E$

F: $A \rightarrow D \rightarrow F$

G: $A \rightarrow I \rightarrow B \rightarrow G$

H: $A \rightarrow H$

I: $A \rightarrow I$

了:

B: $A \rightarrow I \rightarrow B$

$$C: A \rightarrow H \rightarrow C$$
$$D: A \rightarrow D$$
$$E: A \rightarrow I \rightarrow B \rightarrow G \rightarrow E$$
$$F: A \rightarrow D \rightarrow F$$
$$G: A \rightarrow I \rightarrow B \rightarrow G$$
$$H: A \rightarrow H$$
$$I : A \rightarrow I$$

b) Bellman Ford :

9 Passes

1 st	A	B	C	D	E	F	G	H	I	J
	0	∞	∞	∞	∞	∞	∞	∞	∞	∞
		∞	∞	1	∞	4	6	5	-1	1
	0	0	6	1	9	4	6	5	-1	1

2 nd	A	B	C	D	E	F	G	H	I	J
	0	0	6	1	9	4	6	5	-1	1
	0	0	6	-4	9	-1	8	5	-1	1
	0	0	6	-4	4	-1	1	5	-1	1

3 rd	A	B	C	D	E	F	G	H	I	J
	0	0	6	-4	4	-1	1	5	-1	1

No changes after 3rd pass.

Shortest paths :

For A: A
 B: A $\rightarrow$ I $\rightarrow$ B (0)
 C: A $\rightarrow$ H $\rightarrow$ C (6)
 D: A $\rightarrow$ H $\rightarrow$ I $\rightarrow$ D (-4)
 E: A $\rightarrow$ H $\rightarrow$ C $\rightarrow$ D $\rightarrow$ F $\rightarrow$ G $\rightarrow$ E (4)
 F: A $\rightarrow$ H $\rightarrow$ C $\rightarrow$ D $\rightarrow$ F (-1)
 G: A $\rightarrow$ H $\rightarrow$ C $\rightarrow$ D $\rightarrow$ F $\rightarrow$ G (1)
 H: A $\rightarrow$ H (5)

PAPERWORK

Q 4.

Pseudocode

longestCommonSubstring (A, B) :

$n = \text{length}(A)$

$m = \text{length}(B)$

Create table L of size $(n+1) \times (m+1)$

$L[i][j] = 0$

$\text{maxLength} = 0$, $\text{endIndex} = 0$. // where LCS ends in A

for $i = 1$ to n :

for $j = 1$ to m :

// char match if $A[i-1] == B[j-1]$:

// 1 + diagonal $L[i][j] = L[i-1][j-1] + 1$

else

$L[i][j] = 0$

// If longer
substring found

if $L[i][j] > \text{maxLength}$

$\text{maxLength} = L[i][j]$ // update

$\text{endIndex} = i - 1$

// get lcs index
 $\text{startIndex} = \text{endIndex} - \text{maxLen} + 1$

$\text{longestSubstring} = A[\text{startIndex} \dots \text{endIndex}]$

return longestSubstring, maxLen.

Dry Run

		a	b	c	d	e	f	e	f	g	h
		0	0	0	0	0	0	0	0	0	0
b		0	1	0	0	0	0	0	0	0	0
c		0	0	2	0	0	0	0	0	0	0
d		0	0	0	3	0	0	0	0	0	0
e		0	0	0	0	4	0	0	0	0	0
g		0	0	0	0	0	0	0	0	1	0
g		0	0	0	0	0	0	0	0	1	0
f		0	0	0	0	0	1	0	1	0	0
e		0	0	0	0	1	0	2	0	0	0
f		0	0	0	0	0	2	0	3	0	0
g		0	0	0	0	0	0	0	0	4	0
g		0	0	0	0	0	0	0	0	1	0

longest substring : fegfg and bcde

Q 5. counting sum Problem

Pseudocode.

create array ways[0...N]

ways[x] = 0.

// make partitions

for i = 1 to n // i is integer used in partition

for s = 1 to n // s is current sum

ways[s] = ways[s] + ways[s - i]
 return ways[N] - 1 // exclude N = 0

Dry Run

1. Include new coin (copy above value)
2. Include new coin (curr amount - curr coin; copy value from same row)
3. Add 1. and 2.

if $\text{win } i > s/\text{amount}$, copy above value.

Amount / s

	0	1	2	3	4	5	6	7	8	9	10
0	1	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1
2	1	1	¹⁺¹ 2	¹⁺¹ 2	⁴⁺² 3	³⁺² 3	⁴⁺³ 4	³⁺¹ 4	⁴⁺¹ 5	⁴⁺¹ 5	⁵⁺¹ 6
3	1	1	²⁺¹ 2	²⁺¹ 3	³⁺¹ 4	⁵ 5	7	8	10	12	¹⁴⁺¹ 14
4	1	1	2	3	⁴⁺¹ 5	⁵⁺¹ 6	9	11	15	18	²³⁺¹ 23
5	1	1	2	3	5	⁶⁺¹ 7	⁹⁺¹ 10	13	18	23	30
6	1	1	2	3	5	7	¹⁰⁺¹ 11	14	20	26	35
7	1	1	2	3	5	7	11	15	21	28	37
8	1	1	2	3	5	7	11	15	22	29	38
9	1	1	2	3	5	7	11	15	22	30	39
10	1	1	2	3	5	7	11	15	22	30	42

for 3: {0, 1, 2, 3}

^{0, 1, 2, 3}
 3 + ⁰0 = 3

Answer: ways[N] = 42

2 or more +ve integers.

4: {0, 1, 2, 3, 4}

42 - 1 = 41 ways